1. STL (Standard Template Library)
Full Form
STL = Standard Template Library
Definition
The Standard Template Library is a powerful library in C++ that provides predefined classes and functions for common data structures and algorithms such as arrays, vectors, stacks, queues, sorting, and searching.

Components of STL
1. Containers
Store data
Examples:

- vector
- list
- queue
- stack

---

2. Algorithms
Perform operations
Examples:

- sort()
- search()
- reverse()

---

3. Iterators
Used to traverse containers (like pointers)

---

2. Array
Definition
An array is a collection of elements of the same data type, stored in contiguous memory locations, accessed using an index.

Example
int arr[5] = {10, 20, 30, 40, 50};

Key Points

- Fixed size
- Fast access
- Stored in continuous memory

---

3. Dynamic Array
Definition

Athar Ahmed

A dynamic array is an array whose size is determined at runtime using dynamic memory allocation.

Full Form Concept

- DMA = Dynamic Memory Allocation

Example

```
int* arr = new int[n];
```

Key Points

- Size decided at runtime ✓
- Uses pointer ✓
- Requires manual memory management ✓

---

4. Vector

Definition

A vector is a dynamic array provided by the STL, which can automatically resize itself when elements are added or removed.

Example

```
vector<int> v;
v.push_back(10);
```

Key Features

- Auto resizing ✓
- Safe ✓
- Easy to use ✓

---

5. Multidimensional Array

Definition

A multidimensional array is an array of arrays used to store data in tabular form (rows and columns).

Example

```
int arr[2][3] = {
   {1,2,3},
   {4,5,6}
};
```

---

6. Multidimensional Vector

Definition

Athar Ahmed

A multidimensional vector is a vector of vectors used to represent 2D or 3D dynamic data structures.

Example
vector<vector<int>> mat;

Why Use It?

- Dynamic size ✓
- Flexible ✓
- Used in real-world applications ✓

---

7. Pointer
Definition
A pointer is a variable that stores the memory address of another variable.

Example
int x = 10;
int* ptr = &x;

---

8. Memory Management Keywords

Keyword	Full Form	Use
new	—	Allocate memory
delete	—	Free memory

---

9. 2D Dynamic Array Definition
Definition
A 2D dynamic array is a dynamically allocated structure that stores data in rows and columns using pointers.

Example Concept
int** arr;

---

10. Iterator
Definition
An iterator is an object used to point to elements of a container and traverse through them.

Example
vector<int>::iterator it;

---

11. Algorithm (STL)
Definition

Athar Ahmed

An algorithm is a predefined function in STL used to perform operations like sorting, searching, etc.

Example
sort(v.begin(), v.end());

Full Form

Term	Full Form	Definition
STL	Standard Template Library	Library of containers, algorithms, iterators
DMA	Dynamic Memory Allocation	Memory allocation at runtime
Array	—	Fixed-size collection of elements
Vector	—	Dynamic array (auto resizable)
Pointer	—	Stores memory address
Iterator	—	Used to traverse containers

PART 1: What is an Array?
An array stores multiple values of the same type in contiguous memory.
Basic Example (Static Array)

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    for(int i = 0; i < 5; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Problem

- Size is fixed (5)
- Cannot change at runtime ❌

PART 2: Dynamic Arrays (Runtime Memory Allocation)
To overcome fixed size, we use dynamic memory (new)

1D Dynamic Array (Beginner Level)

```
#include <iostream>
using namespace std;
```

Attar Ahmed

```
int main() {
    int n;

    cout << "Enter size: ";
    cin >> n;

    int* arr = new int[n]; // dynamic array

    // Input
    for(int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // Output
    for(int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    delete[] arr; // free memory

    return 0;
}
```

Important Concepts

- new → allocates memory
- delete[] → releases memory
- Pointer (int* arr) stores base address

Common Mistake

Forgetting delete[] arr → causes memory leak

PART 3: 2D Dynamic Array

Now we create a matrix dynamically.

2D Dynamic Array Code

```
#include <iostream>
using namespace std;

int main() {
    int rows, cols;

    cout << "Enter rows and cols: ";
    cin >> rows >> cols;
```

Athar Ahmed

```
    int** arr = new int*[rows];

    for(int i = 0; i < rows; i++) {
        arr[i] = new int[cols];
    }

    // Input
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            cin >> arr[i][j];
        }
    }

    // Output
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }

    // Free memory
    for(int i = 0; i < rows; i++) {
        delete[] arr[i];
    }
    delete[] arr;

    return 0;
}
```

PART 4: Introduction to Vectors (Modern C++)
Vectors are dynamic arrays provided by STL.

Basic Vector Example

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {10, 20, 30};

    v.push_back(40); // add element
```

Athar Ahmed

```
    for(int i = 0; i < v.size(); i++) {
        cout << v[i] << " ";
    }

    return 0;
}
```

Why Vectors?

- Automatically resize ✓
- No need for delete ✓
- Safer ✓

PART 5: 2D Vectors (Multidimensional)

Create 2D Vector

```
vector<vector<int>> mat(3, vector<int>(4, 0));
```

3 rows, 4 columns initialized with 0

Full Example

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int rows = 2, cols = 3;

    vector<vector<int>> mat(rows, vector<int>(cols));

    // Input
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            cin >> mat[i][j];
        }
    }

    // Output
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            cout << mat[i][j] << " ";
        }
        cout << endl;
    }
```

Athar Ahmed

```
    return 0;
}
```

PART 6: Dynamic 2D Vector (Flexible Rows)

Example (Jagged Array)

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<vector<int>> mat;

    mat.push_back({1, 2});
    mat.push_back({3, 4, 5}); // different size

    for(int i = 0; i < mat.size(); i++) {
        for(int j = 0; j < mat[i].size(); j++) {
            cout << mat[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

PART 7: 3D Vector (Advanced)

Example

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<vector<vector<int>>> cube(2,
        vector<vector<int>>(3, vector<int>(4, 1)));

    cout << cube[0][0][0];

    return 0;
}
```

Athar Ahmed

PART 8: STL Functions (Important)

Useful Operations

```
vector<int> v;

v.push_back(10);
v.pop_back();
v.size();
v.clear();
v.empty();
```

PART 9: Interview Comparison

Feature	Dynamic Array	Vector
Memory	Manual	Automatic
Resize	❌	✓
Safety	❌	✓
Use in real projects	Rare	Very Common

PART 10: Real-World Example

Store Student Marks (2D Vector)

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int students = 2, subjects = 3;

    vector<vector<int>> marks(students, vector<int>(subjects));

    // Input
    for(int i = 0; i < students; i++) {
        for(int j = 0; j < subjects; j++) {
            cin >> marks[i][j];
        }
    }

    // Average per student
    for(int i = 0; i < students; i++) {
        int sum = 0;
        for(int j = 0; j < subjects; j++) {
            sum += marks[i][j];
        }
        cout << "Avg of student " << i << " = "
```

Athar Ahmed

```
            << (sum / subjects) << endl;
    }

    return 0;
}
```

FINAL SUMMARY

- Array → Fixed size
- Dynamic Array → Runtime allocation (manual memory)
- Vector → Best choice (automatic + flexible)
- 2D/3D Vector → Used in real-world apps

---

1. Multidimensional Vectors (C++)

A multidimensional vector is basically a vector of vectors (like a dynamic 2D or 3D array).

2D Vector Syntax

```
vector<vector<int>> matrix;
```

Initialize with size

```
vector<vector<int>> matrix(3, vector<int>(4, 0));
```

Creates a 3×4 matrix filled with 0

Example: Input & Display 2D Vector

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int rows = 2, cols = 3;

    vector<vector<int>> mat(rows, vector<int>(cols));

    // Input
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            cin >> mat[i][j];
        }
    }

    // Output
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            cout << mat[i][j] << " ";
        }
```

Athar Ahmed

```
        cout << endl;
    }

    return 0;
}
```

Dynamic Row Addition

```
vector<vector<int>> mat;

mat.push_back({1,2,3});
mat.push_back({4,5,6});
```

3D Vector Example

```
vector<vector<vector<int>>> cube(2, vector<vector<int>>(3, vector<int>(4, 1)));
```

---

2. Dynamically Created Arrays (C++)

Dynamic arrays are created at runtime using pointers.

1D Dynamic Array

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int* arr = new int[n];

    for(int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    for(int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    delete[] arr; // Free memory

    return 0;
}
```

2D Dynamic Array

```
#include <iostream>
```

Athar Ahmed

```
using namespace std;

int main() {
    int rows = 2, cols = 3;

    int** arr = new int*[rows];

    for(int i = 0; i < rows; i++) {
        arr[i] = new int[cols];
    }

    // Input
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            cin >> arr[i][j];
        }
    }

    // Output
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }

    // Free memory
    for(int i = 0; i < rows; i++) {
        delete[] arr[i];
    }
    delete[] arr;

    return 0;
}
```

Key Difference (Interview Important)

Feature	Vector	Dynamic Array
Memory management	Automatic	Manual (new/delete)
Resizable	Yes	No (fixed after creation)
Safety	High (bounds checking with .at())	Low
Ease of use	Easy	Complex

When to Use What?

Athar Ahmed

- Use vector In 90% cases (recommended)
- Use dynamic arrays When:
    - You need low-level memory control
    - Working with legacy C-style code
    - Performance-critical systems

Athar Ahmed